

Lesson 3 — Connect FastMCP to the LangChain order assistant

Goal: connect the FastMCP order service from Lesson 2 to the OpenAI/LangChain assistant from Lesson 1 as a real tool-calling flow.

We will connect the pieces in this order:

1. Start the FastMCP HTTP server
2. Discover its tools with `MultiServerMCPClient`
3. Bind those tools to the OpenAI model
4. Let the model request an MCP tool
5. Execute the request through MCP
6. Return a validated `SupportAnswer`

Lesson 1 owns the assistant. Lesson 2 owns the order service. This lesson owns the connection.

0. Setup

We use the stable `langchain-mcp-adapters` package. Its `MultiServerMCPClient` discovers remote MCP capabilities and exposes them as normal LangChain tools without importing server functions.

```
In [3]: # Run once if needed
%pip install -qU langchain-core==1.6.3 langchain-mcp-adapters==0.3.2 langcha
```

Note: you may need to restart the kernel to use updated packages.

Load the OpenAI key

As in Lesson 1, the key must be stored as `OPENAI_API_KEY` in a `.env` file beside the notebook. Setup fails immediately if the file or key is missing.

```
In [4]: from pathlib import Path

from dotenv import dotenv_values, load_dotenv

dotenv_path = Path.cwd() / ".env"
if not dotenv_path.is_file():
    raise FileNotFoundError(
        f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
    )

env_values = dotenv_values(dotenv_path)
```

```

if not env_values.get("OPENAI_API_KEY"):
    raise RuntimeError(
        f"OPENAI_API_KEY is missing or empty in {dotenv_path}."
    )

load_dotenv(dotenv_path, override=True)

MCP_URL = "http://127.0.0.1:8000/mcp"
SERVER_FILE = Path.cwd() / "order_mcp_server.py"
if not SERVER_FILE.is_file():
    raise FileNotFoundError(f"Missing Lesson 2 server: {SERVER_FILE}")

```

1. The completed architecture

User question

- Lesson 1 prompt + OpenAI model
- model requests get_order_status
- LangChain MultiServerMCPClient
- Streamable HTTP
- Lesson 2 FastMCP server
- validated order result
- ToolMessage returned to model
- validated SupportAnswer

The model sees a normal tool schema. It does not need to know about HTTP, FastMCP, or the order database.

2. Start the Lesson 2 server

Open a terminal in this folder and keep the server running:

```
python order_mcp_server.py
```

It exposes a Streamable HTTP MCP endpoint at `http://127.0.0.1:8000/mcp`. The following notebook cells will fail clearly if the server is not reachable.

```
In [14]: %cat order_mcp_server.py
```

```

"""FastMCP order service shared by Lessons 2 and 3."""

import os
from typing import Literal

from fastmcp import FastMCP
from pydantic import BaseModel, Field

ORDER_DATABASE = {
    "A100": "shipped",
    "B200": "processing",
}

class OrderStatus(BaseModel):
    """Validated order-status result returned by the MCP tool."""

    order_id: str = Field(description="Order identifier supplied by the use  
r")
    status: Literal["shipped", "processing", "not_found"]

order_mcp = FastMCP(
    "Order Service",
    instructions="Use get_order_status for factual order-status lookups.",
)

@order_mcp.tool
def get_order_status(order_id: str) -> OrderStatus:
    """Return the trusted status for one order."""
    status = ORDER_DATABASE.get(order_id, "not_found")
    return OrderStatus(order_id=order_id, status=status)

@order_mcp.resource("orders://policy")
def order_policy() -> str:
    """Explain the meanings of order statuses."""
    return (
        "shipped: handed to carrier; "  

        "processing: being prepared; "  

        "not_found: no matching order"
    )

@order_mcp.prompt
def investigate_order(order_id: str) -> str:
    """Create a reusable request for an order investigation."""
    return (
        f"Check order {order_id} with get_order_status, then explain "  

        "the result clearly without inventing missing details."
    )

if __name__ == "__main__":

```

```

order_mcp.run(
    transport="http",
    host=os.getenv("MCP_HOST", "127.0.0.1"),
    port=int(os.getenv("MCP_PORT", "8000")),
)

```

3. Discover the MCP tool as a LangChain tool

`MultiServerMCPClient` connects, performs protocol discovery, and converts the advertised MCP tool into a standard LangChain tool. No client-side order schema is copied by hand.

```

In [15]: from langchain_mcp_adapters.client import MultiServerMCPClient

mcp_client = MultiServerMCPClient({
    "orders": {"transport": "http", "url": MCP_URL}
})
discovered_tools = await mcp_client.get_tools()

for discovered_tool in discovered_tools:
    print(discovered_tool.name)
    print(discovered_tool.description)
    schema = discovered_tool.args_schema
    print(schema if isinstance(schema, dict) else schema.model_json_schema())

```

`get_order_status`

Return the trusted status for one order.

```

{'properties': {'order_id': {'type': 'string'}}, 'required': ['order_id'],
'type': 'object', 'additionalProperties': False}

```

4. OpenAI tool calling through MCP

The flow is still the Lesson 1 tool loop. Only the tool implementation changed:

- Before: LangChain called a local Python function.
- Now: LangChain calls an adapted tool that sends an MCP request to the FastMCP server.

```

In [16]: from typing import Literal

from langchain_core.messages import HumanMessage, ToolMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class SupportAnswer(BaseModel):
    answer: str
    status: Literal["shipped", "processing", "not_found", "unknown"]
    grounded: bool = Field(

```

```

        description="True only when the status comes from a successful MCP t
    )

support_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        """
You are a minimal order-support assistant.
Use get_order_status whenever an order status is requested.
Never invent a status. Base the final answer only on a successful tool result.
If verified information is unavailable, use status='unknown'.
        """,
    ),
    ("human", "{question}"),
])

MODEL = "gpt-5.6-luna"
llm = ChatOpenAI(
    model=MODEL,
    use_responses_api=True,
)
final_model = llm.with_structured_output(SupportAnswer).with_retry(
    stop_after_attempt=3
)

```

One complete request, with the plumbing visible

Invoking an adapted tool with the model's full tool-call object returns a correctly matched `ToolMessage`. Structured MCP output is also retained as the message's artifact.

```

In [17]: question = "Where is order B200?"
messages = support_prompt.invoke({"question": question}).to_messages()

mcp_tools = await mcp_client.get_tools()
tool_map = {available_tool.name: available_tool for available_tool in mcp_tools}
model_with_tools = llm.bind_tools(mcp_tools).with_retry(stop_after_attempt=3)

# 1. The model requests a tool; it does not execute it.
first_response = await model_with_tools.ainvoke(messages)
messages.append(first_response)
print("Requested:", first_response.tool_calls)

# 2. Application code executes each request through MCP.
for call in first_response.tool_calls:
    selected_tool = tool_map.get(call["name"])
    if selected_tool is None:
        messages.append(
            ToolMessage(
                content="Tool unavailable",
                tool_call_id=call["id"],
                status="error",
            )
        )

```

```

    )
    continue

    tool_message = await selected_tool.ainvoke(call)
    messages.append(tool_message)
    print("MCP result:", tool_message.content)
    print("MCP artifact:", tool_message.artifact)

# 3. The model converts verified tool evidence into our application schema.
    messages.append(
        HumanMessage(
            """
Return the final structured answer using only successful tool results above.
If there is no successful result, use status='unknown' and grounded=False.
Map the MCP status 'not_found' directly to status='not_found'.
            """
        )
    )

    final_answer = await final_model.ainvoke(messages)
    final_answer

```

Requested: [{'name': 'get_order_status', 'args': {'order_id': 'B200'}, 'id': 'call_GLXH9y1nDoUFGCZATPJVMDH6', 'type': 'tool_call'}]
MCP result: [{'type': 'text', 'text': '{"order_id": "B200", "status": "processing"}', 'id': 'lc_40c34fde-431f-4cc7-8e40-3c3418f5e161'}]
MCP artifact: {'structured_content': {'order_id': 'B200', 'status': 'processing'}}

Out[17]: SupportAnswer(answer='Order B200 is processing.', status='processing', grounded=True)

5. Put the integration into one function

The client-created LangChain tools open MCP sessions when invoked, so application code does not need a custom async context manager. Server-reported tool errors become error `ToolMessage` objects; network or transport failures still raise because the model cannot repair a broken connection.

```

In [18]: async def ask_order_assistant(question: str) -> SupportAnswer:
    messages = support_prompt.invoke({"question": question}).to_messages()

    tools = await mcp_client.get_tools()
    tool_map = {available_tool.name: available_tool for available_tool in tools}
    model_with_tools = llm.bind_tools(tools).with_retry(stop_after_attempt=3)

    first_response = await model_with_tools.ainvoke(messages)
    messages.append(first_response)

    for call in first_response.tool_calls:
        selected_tool = tool_map.get(call["name"])
        if selected_tool is None:
            messages.append(
                ToolMessage(

```

```
        content="Tool unavailable",
        tool_call_id=call["id"],
        status="error",
    )
    )
    continue

    messages.append(await selected_tool.ainvoke(call))

    messages.append(
        HumanMessage(
            """
Return the final structured answer using only successful tool results above.
If no successful result exists, use status='unknown' and grounded=False.
Map the MCP status 'not_found' directly to status='not_found'.
"""
        )
    )

    return await final_model.ainvoke(messages)
```

```
In [19]: print(await ask_order_assistant("What is happening with order A100?"))
print(await ask_order_assistant("What is happening with order XYZ?"))
```

answer='Order A100 has shipped.' status='shipped' grounded=True
answer='Order XYZ was not found.' status='not_found' grounded=True

6. What each layer is responsible for

Layer	Responsibility
OpenAI model	Decide when and how to call a tool; produce the final answer
LangChain	Prompting, tool-call messages, retries, structured final output
MultiServerMCPClient	Discover MCP tools and expose them as LangChain tools
MCP	Standardize discovery, invocation, results, and transport
FastMCP server	Validate inputs, run trusted order logic, return typed data
Pydantic	Enforce server and application data contracts

Do not retry every failure blindly: model API failures, MCP transport failures, MCP tool errors, and invalid business data are different failure classes.

7. Complete client code overview

This final cell contains all Lesson 3 client code together. The FastMCP server remains in `order_mcp_server.py` and must be running first.

```

In [20]: from pathlib import Path
         from typing import Literal

         from dotenv import dotenv_values, load_dotenv
         from langchain_core.messages import HumanMessage, ToolMessage
         from langchain_core.prompts import ChatPromptTemplate
         from langchain_openai import ChatOpenAI
         from langchain_mcp_adapters.client import MultiServerMCPClient
         from pydantic import BaseModel, Field

         # 1. Require the .env key and Lesson 2 server file.
         dotenv_path = Path.cwd() / ".env"
         if not dotenv_path.is_file():
             raise FileNotFoundError(
                 f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
             )
         env_values = dotenv_values(dotenv_path)
         if not env_values.get("OPENAI_API_KEY"):
             raise RuntimeError(f"OPENAI_API_KEY is missing or empty in {dotenv_path}")
         load_dotenv(dotenv_path, override=True)

         server_file = Path.cwd() / "order_mcp_server.py"
         if not server_file.is_file():
             raise FileNotFoundError(f"Missing Lesson 2 server: {server_file}")
         MCP_URL = "http://127.0.0.1:8000/mcp"
         mcp_client = MultiServerMCPClient({
             "orders": {"transport": "http", "url": MCP_URL}
         })

         # 2. Final application contract.
         class SupportAnswer(BaseModel):
             answer: str
             status: Literal["shipped", "processing", "not_found", "unknown"]
             grounded: bool = Field(
                 description="True only when the status comes from a successful MCP t
             )

         # 3. Prompt and OpenAI models from Lesson 1.
         support_prompt = ChatPromptTemplate.from_messages([
             (
                 "system",
                 """
                 You are a minimal order-support assistant.
                 Use get_order_status whenever an order status is requested.
                 Never invent a status. Base the final answer only on a successful tool result.
                 If verified information is unavailable, use status='unknown'.
                 """,
             ),
             ("human", "{question}"),
         ])

         llm = ChatOpenAI(

```

```

    model="gpt-5.6-luna",
    use_responses_api=True,
)
final_model = llm.with_structured_output(SupportAnswer).with_retry(
    stop_after_attempt=3
)

# 4. Discover MCP tools, let the model request one, and execute it.
async def ask_order_assistant(question: str) -> SupportAnswer:
    messages = support_prompt.invoke({"question": question}).to_messages()

    tools = await mcp_client.get_tools()
    tool_map = {available_tool.name: available_tool for available_tool in tools}
    model_with_tools = llm.bind_tools(tools).with_retry(stop_after_attempt=3)

    first_response = await model_with_tools.ainvoke(messages)
    messages.append(first_response)

    for call in first_response.tool_calls:
        selected_tool = tool_map.get(call["name"])
        if selected_tool is None:
            messages.append(
                ToolMessage(
                    content="Tool unavailable",
                    tool_call_id=call["id"],
                    status="error",
                )
            )
            continue
        messages.append(await selected_tool.ainvoke(call))

    messages.append(
        HumanMessage(
            """
Return the final structured answer using only successful tool results above.
If no successful result exists, use status='unknown' and grounded=False.
Map the MCP status 'not_found' directly to status='not_found'.
"""
        )
    )
    return await final_model.ainvoke(messages)

# 5. Examples. Start order_mcp_server.py in a separate terminal first.
print(await ask_order_assistant("What is happening with order A100?"))
print(await ask_order_assistant("What is happening with order XYZ?"))

```

```

answer='Order A100 has shipped.' status='shipped' grounded=True
answer='Order XYZ was not found.' status='not_found' grounded=True

```

8. Learning checkpoint

You now have a complete custom MCP integration if you can explain:

1. How `MultiServerMCPClient` turns a remote MCP capability into a LangChain tool.
2. Why the model requests the tool but application code executes it.
3. Why the `ToolMessage` must use the original tool-call ID.
4. Where validation happens on the server and in the final application output.
5. Why a transport failure should not be treated like a recoverable tool error.

References

- [LangChain MCP adapters](#)
- [FastMCP documentation](#)
- [OpenAI GPT-5.6 Luna](#)
- [MCP architecture](#)